DOCUMENTARY PROMPT ANSWERS

This documentary reference now tells the story of the Myntra-style store after the modification ideas were implemented. Each section explains what the experience looked like before and how the new version changes the narrative.

1. ARCHITECTURE NARRATIVE

Before the changes, the documentary arc was about a clean React frontend paired with a lightweight Express backend that returned formatted prices and a banner. It showed a storefront that was visually polished but mostly passive: the user could browse products, see categories, and understand how the API response powered the grid.

After the changes, the story becomes more dynamic. The business need is no longer just "show curated festival-ready products," but "help shoppers explore, filter, save, and act on those products without losing the simplicity of the codebase." The challenge becomes orchestration: one API payload now needs to support rotating promotions, hover-led exploration, product filtering, and a persistent mini cart. The resolution is still the React UI plus Express catalog pairing, but now the frontend turns one curated response into multiple merchandised experiences, and the backend has evolved from a static route into a small, observable catalog service backed by JSON data and validation.

2. TECHNICAL PERSONAS

DESIGNER

Before the changes, the designer mainly shaped the hero, category cards, and product grid in ‘frontend/src/App.css‘. The visual system was already warm and commerce-oriented, but the motion and interactivity were restrained.

After the changes, the designer’s scope expands to:

- the animated ‘OfferRotator‘ panel that replaced the old status card
- the mega menu dropdown states and grouped storytelling blocks
- the filter drawer layout and form controls
- the mini cart sidebar and saved-state presentation

The key files remain ‘frontend/src/App.css‘ and ‘frontend/src/index.css‘, but the design work now covers motion, layered panels, and responsive multi-column composition rather than just static sections.

FRONT-END ENGINEER

Before the changes, the front-end engineer mostly owned ‘fetchCatalog()‘, the loading/error states, and mapping ‘products‘ into ‘ProductCard‘ components in ‘frontend/src/App.jsx‘.

After the changes, the front-end engineer now owns several coordinated systems inside that same file:

- ‘MegaMenu‘ for hover and focus interactions with documented ARIA attributes
- ‘FilterDrawer‘ for category, price, and delivery selectors
- ‘OfferRotator‘ for timer-based banner rotation
- ‘MiniCart‘ for ‘localStorage‘ persistence, remove actions, and running totals
- derived collections such as ‘megaMenuGroups‘, ‘filteredProducts‘, and ‘cartTotal‘

The frontend role is now less about rendering one payload directly and more about deriving multiple user experiences from shared state.

BACKEND ENGINEER

Before the changes, the backend engineer owned one inline array in ‘backend/index.js‘, one route, one startup log, and server-side price formatting.

After the changes, the backend engineer owns a more maintainable API boundary:

- catalog content moved into ‘backend/catalog.json‘
- response shaping still happens in ‘backend/index.js‘
- request logging middleware records live traffic
- ‘validateCatalogResponse()‘ guards the response contract
- ‘GET /api/health‘ offers a basic operational endpoint
- ‘validate-catalog.js‘ provides a script-based integrity check for the source data

The backend persona now reads less like a demo setup and more like an engineer preparing a small service for growth.

3. DATA PIPELINE VOICEOVER

Before the modifications, the server took an inline product array, formatted each INR price, and returned a banner plus a curated list. The frontend fetched that response on first render, saved it into state, and used it to populate the hero highlights and product cards.

After the modifications, the backend reads catalog content from ‘backend/catalog.json‘, enriches products with ‘priceFormatted‘, validates the payload, and sends a response that includes both ‘offers‘ and ‘curated‘ products. The frontend in ‘frontend/src/App.jsx‘ hydrates that response into multiple layers at once: the banner chips, the rotating localized offer panel, the mega menu groupings, the filter drawer

results, and the localStorage-backed mini cart. The pipeline is still compact, but it now feels like a fuller retail system because one response powers discovery, promotion, and persistence.

## 4. OBSERVABILITY CUTAWAYS

Before the changes, the best observability moment was the hero status card and the ‘console.error(error)‘ path inside ‘fetchCatalog()‘. On the backend, the main signal was the startup log emitted when the server began listening.

After the changes, the documentary has better cutaways to highlight:

- the rotating offer panel, which shows that promotional messages are now part of the live payload
- the filter drawer result count, which makes frontend-derived state visible
- the mini cart, which demonstrates that state persists between sessions through ‘localStorage‘
- request logging middleware in ‘backend/index.js‘, which now prints method, path, status, and duration for every request
- the ‘/api/health‘ endpoint as a lightweight operational probe
- ‘npm run validate‘ as a simple data integrity gate for the JSON-backed catalog

The repo is still intentionally small, but the observability story is now much stronger than before because the backend emits per-request traces and the frontend exposes more of its state transitions to the viewer.

## 5. NEXT EXPERIMENTS

Before the modifications, the next experiments centered on adding tests, logging, and perhaps a database-backed catalog. Those ideas still matter, but the new codebase opens better follow-on paths.

After the modifications, the most useful next experiments are:

- add automated UI tests for the filter drawer, mega menu, and mini cart persistence
- add API tests that verify ‘offers‘, ‘curated‘, and ‘priceFormatted‘ together
- move from ‘catalog.json‘ to a mock database or service layer without changing the response contract
- record analytics for filter selection, menu interaction, and cart additions
- add a small badge or toast system so users get feedback when cart state changes

The important documentary point is that the app now has real interaction seams to explore, not just static rendering seams.